



Non-Human Identities Need Access, Too

Learn how Tailscale and PAM work together to secure pipelines, service accounts, and AI agents.

DATA SCIENCE

Understanding the Mathematics behind Gradient Descent.

A simple mathematical intuition behind one of the optimisation algorithms in Machine Learning.

Parul Pandey

Mar 18, 2019 10 min read

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article



PC: Pexels/Pixabay

"Premature optimization is the root of all e
Ervin Knuth

Agile is a pretty well-known term in the software development process. The basic idea behind it is simple: build something quickly, ➡ get it out there, ➡ get some feedback ➡ make changes depending upon the feedback ➡ repeat the process. The **goal** is to get the product near the user and guide you with **feedback** to obtain the best possible product with the **least error**. Also, the **steps** taken for improvement need to be small and should constantly involve, an Agile software development process iterations. The idea of – start with a solution possible, measure and iterate as frequently as possible, Gradient descent under the hood.

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Objective

Gradient descent algorithm is an iterative process used to find the minimum of a function (barring some constraints). The formula below sums up the entire Gradient Descent algorithm in a single line.

The diagram shows the formula $\Theta^1 = \Theta^0 - \alpha \nabla J(\Theta)$ with the word "eval" to its right. Callouts explain the components:

- Θ^1 : next position (red bubble)
- Θ^0 : current position (blue bubble)
- α : small step (green bubble)
- $\nabla J(\Theta)$: direction of fastest increase (purple bubble)
- The minus sign: opposite direction (black bubble)

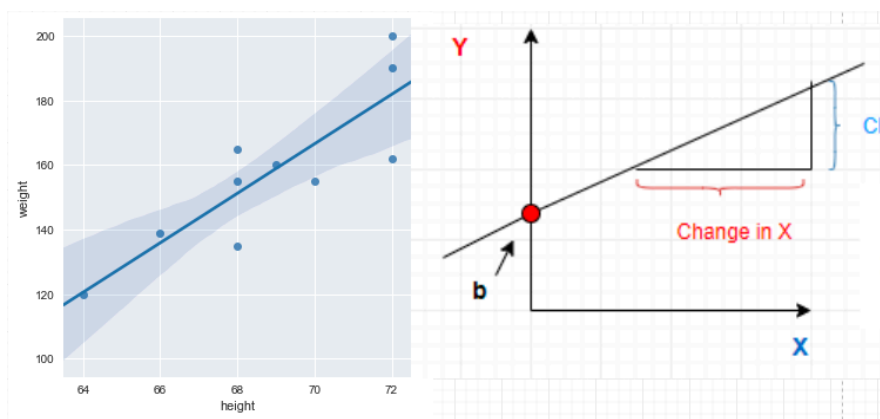
<https://www.coursehero.com/file/27927651/Gradient-De>

But how do we arrive at this formula? Well, it's straightforward and includes some high school mathematics. Through this article, we shall try to understand this formula in the context of a Linear Regression.

This article is an adaption of the video titled **Mathematics of Gradient Descent – Intelligence and Learning by The Coding Train**. This article was initially created as notes to supplement my understanding. I'll highly recommend to see the video too.

A Machine Learning Model

- Consider a bunch of data points in a 2 D that the data is related to the height and group of students. We are trying to predict relationship between these quantities to weight of some new students afterward. a simple example of a supervised Machine technique.
- Let us now draw an arbitrary line in space through some of these data points. The equation of a straight line would be $Y = mX + b$ where m is its slope and b is its intercept on the Y-axis.

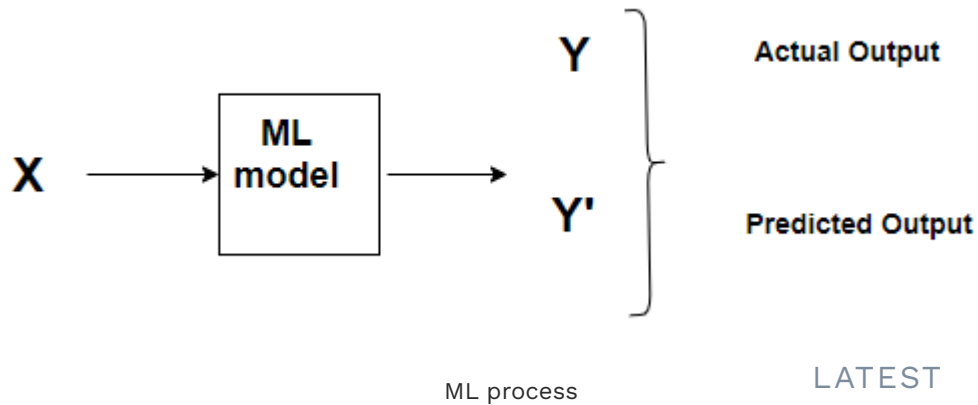


Predictions

Given a known set of inputs and their corresponding outputs, a machine learning model tries to make some predictions.

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)

a new set of inputs.



The Error would be the difference between the actual and predicted outputs.

$$\text{Error} = Y'(\text{Predicted}) - Y(\text{Actual})$$

This relates to the idea of a **Cost function** or

Cost Function

A **Cost Function/Loss Function** evaluates the performance of our Machine Learning Algorithm. The **Loss function** measures the error for a single training example, while the **Cost function** is the average of the loss functions for all the training examples. Henceforth, I shall be using both terms interchangeably.

A Cost function basically tells us 'how good our model is at making predictions for a given value of parameters'.

Let's say there are a total of 'N' points in the dataset. For all those 'N' data points, we want to minimize the Cost function. The Cost function would be the total squared error.

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

$$Cost = \frac{1}{N} \sum_{i=1}^N (Y' - Y)^2$$

The cost function for N data points

Why do we take the squared differences and simply not the absolute differences? Because the squares make it easier to derive a regression line. In that line we need to compute the first derivative of the Cost function, and it is much harder to compute the derivative of absolute values than squared values. Squared differences increase the error distribution, making the bad predictions more pronounced than the good ones.

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)

Minimizing the Cost Function

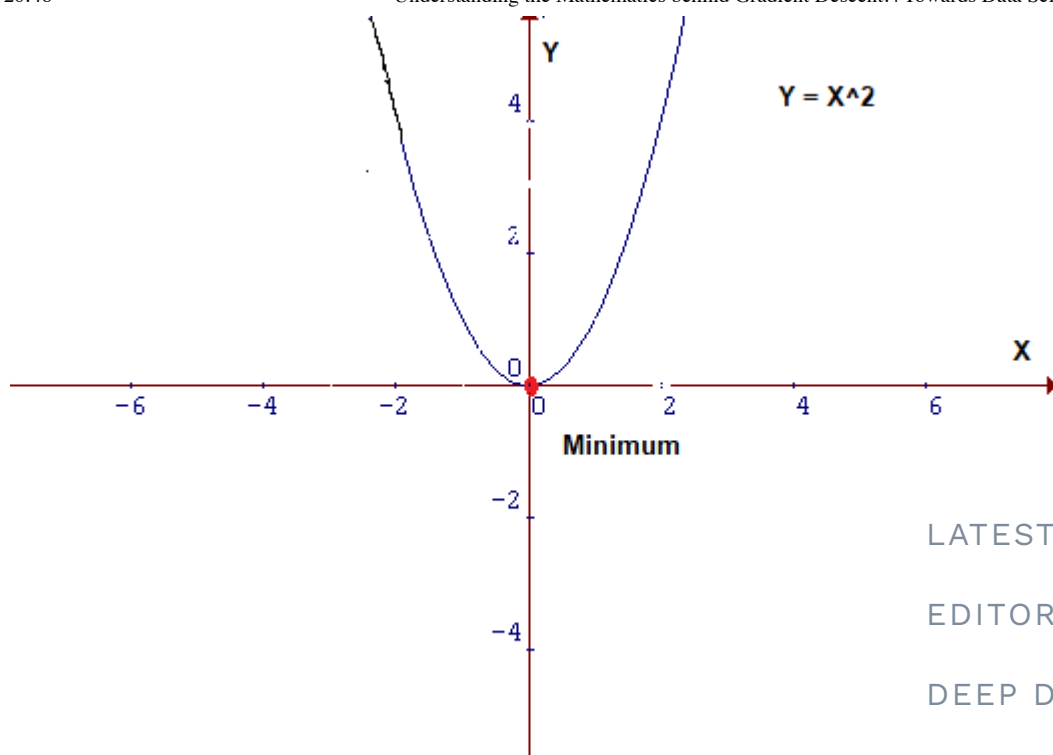
[Submit an Article](#)

The goal of any Machine Learning Algorithm is to minimize the Cost Function.

This is because a lower error between the actual and predicted values signifies that the algorithm has done an excellent job learning. Since we want the lowest possible error, we want those 'm' and 'b' values that give the lowest possible error.

How do we minimize any function?

If we look carefully, our **Cost function** is of the form of a parabola. In a Cartesian coordinate system, this is an upward-opening parabola and can be graphically represented



Parabola

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

To minimize the function above, we need to find **x that produces the lowest value of y** which is pretty easy to locate the minima here since but this may not always be the case, especially in high dimensions. For those cases, we need to devise an algorithm to locate the minima, and that algorithm is called **Gradient Descent**.

Gradient Descent

Gradient descent is one of the most popular optimization algorithms and is the most common way to optimize neural networks. It is an iterative optimization algorithm used to find the minimum value of a function.

Intuition

Consider that you are walking along with the graph below, and you are currently at the '**green**' dot. You aim to reach the minimum, i.e., the '**red**' dot, but from your position, you are unable to view it.

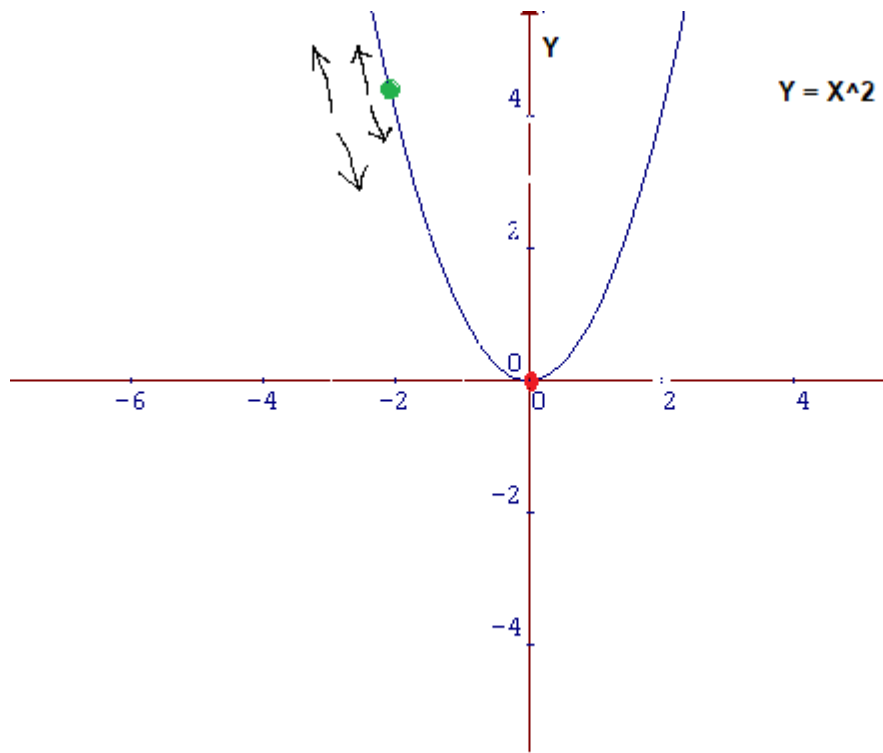


Figure 2

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)

Possible actions would be:

- You might go upward or downward
- If you decide which way to go, you might step or a little step to reach your destination

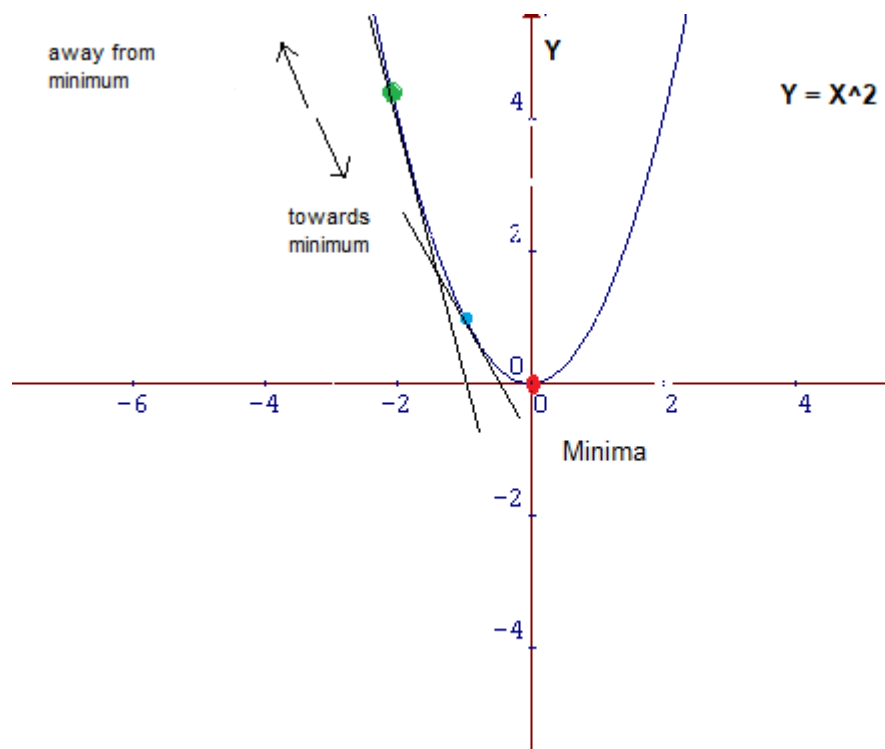
Essentially, there are two things that you should consider to reach the minima, i.e. which way to go and how many steps to take.

Gradient Descent Algorithm helps us to make decisions efficiently and effectively with the use of derivatives. The term **derivative** is a term that comes from calculus, which is calculated as the slope of the graph at a particular point.

slope is described by drawing a tangent line to the graph at the point. So, if we can compute this tangent line, we might compute the desired direction to reach the minima. We will talk about this in more detail in the latter part of the article.

The Minimum Value

In the same figure, if we draw a tangent at the green point, we know that if we are moving upwards, we are moving away from the minima and vice versa. Also, the tangent line indicates the sense of the steepness of the slope.



The slope at the blue point is less steep than at the green point, which means it will take much longer to reach the minimum from the blue point than from the green point.

Mathematical Interpretation of Cost Function

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Let us now put all these learnings into a mathematical formula. In the equation, $y = mx+b$ 'm' and 'b' are its parameters. During the training process, there will be a small change in their values. Let that small change be denoted by δ . The value of parameters will be updated as **$m=m-\delta m$** and **$b=b-\delta b$** , respectively. Our aim here is to find those values of m and b in $y = mx+b$ for which the error is minimum, i.e., values that minimize the cost function.

Rewriting the cost function:

Parameters with small changes:

$m =$ NEWSLETTER
 $b =$

Given Cost Function for 'N' no of samples

$$Cost = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Sign in

Submit an Article

Cost function is denoted by J where J is a function of m and b

$$J_{m,b} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Substituting the term $\hat{y}_i - y_i$ with error for simplicity

$$J_{m,b} = \frac{1}{N} \sum_{i=1}^N e_i^2$$

The idea is that by being able to compute the derivative/slope of the function, we can find the minimum of a function.

The Learning rate

This size of steps taken to reach the minimum or bottom is called **Learning Rate**. We can cover more area with larger steps/higher learning rate but are at the risk of overshooting the minima. On the other hand, small steps/smaller learning rates will consume a lot of time to reach the lowest point.

The visualizations below give an idea about the Learning Rate concept. **See how in the third figure, we reach the minimum point with the minimum number of steps. The learning rate for this problem.**

Set learning rate:

0.1

Execute single step:

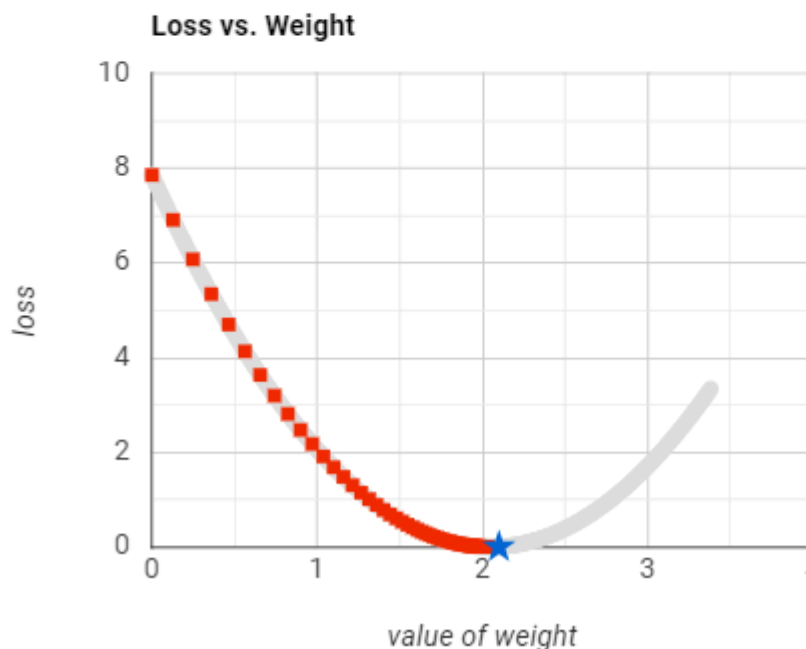
STEP

81

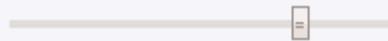
Reset the graph:

RESET

✓ This model has reached minimal loss. Is it possible to achieve similar results with a different learning rate?

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Set learning rate:



1.00

Execute single step:

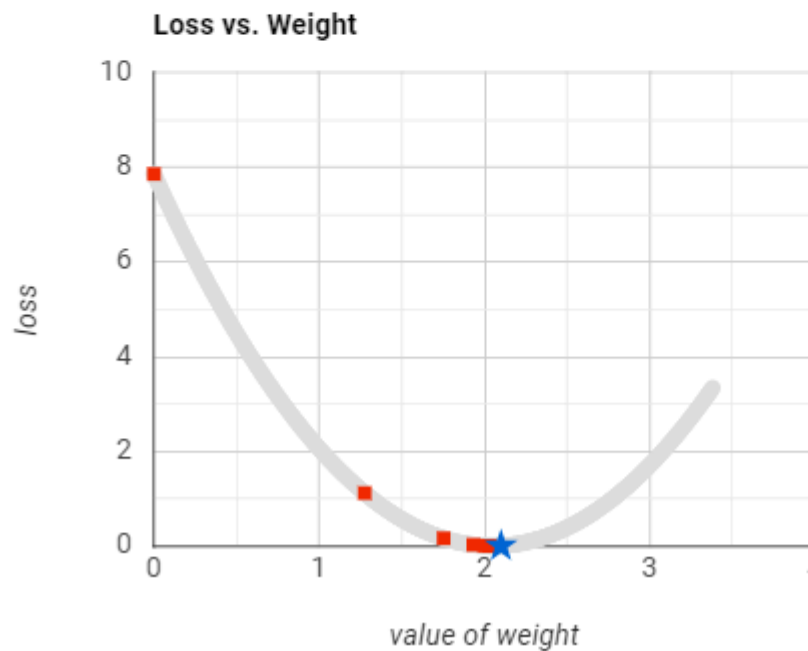
STEP

6

Reset the graph:

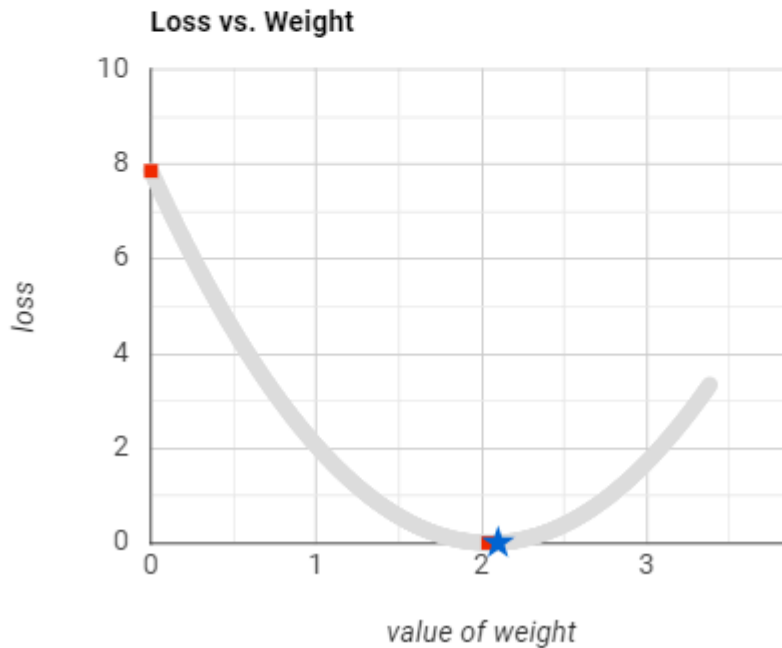
RESET

✓ This model has reached minimal loss. Is it possible to achieve similar results with fewer steps?

[LATEST](#)[EDITOR'S PICKS](#)[DEEP DIVES](#)[NEWSLETTER](#)[WRITE FOR TDS](#)[Sign in](#)[Submit an Article](#)

Set learning rate:	<input type="range" value="1.60"/>	1.60
Execute single step:	<button>STEP</button>	1
Reset the graph:	<button>RESET</button>	

✓ This model has reached minimal loss.



LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

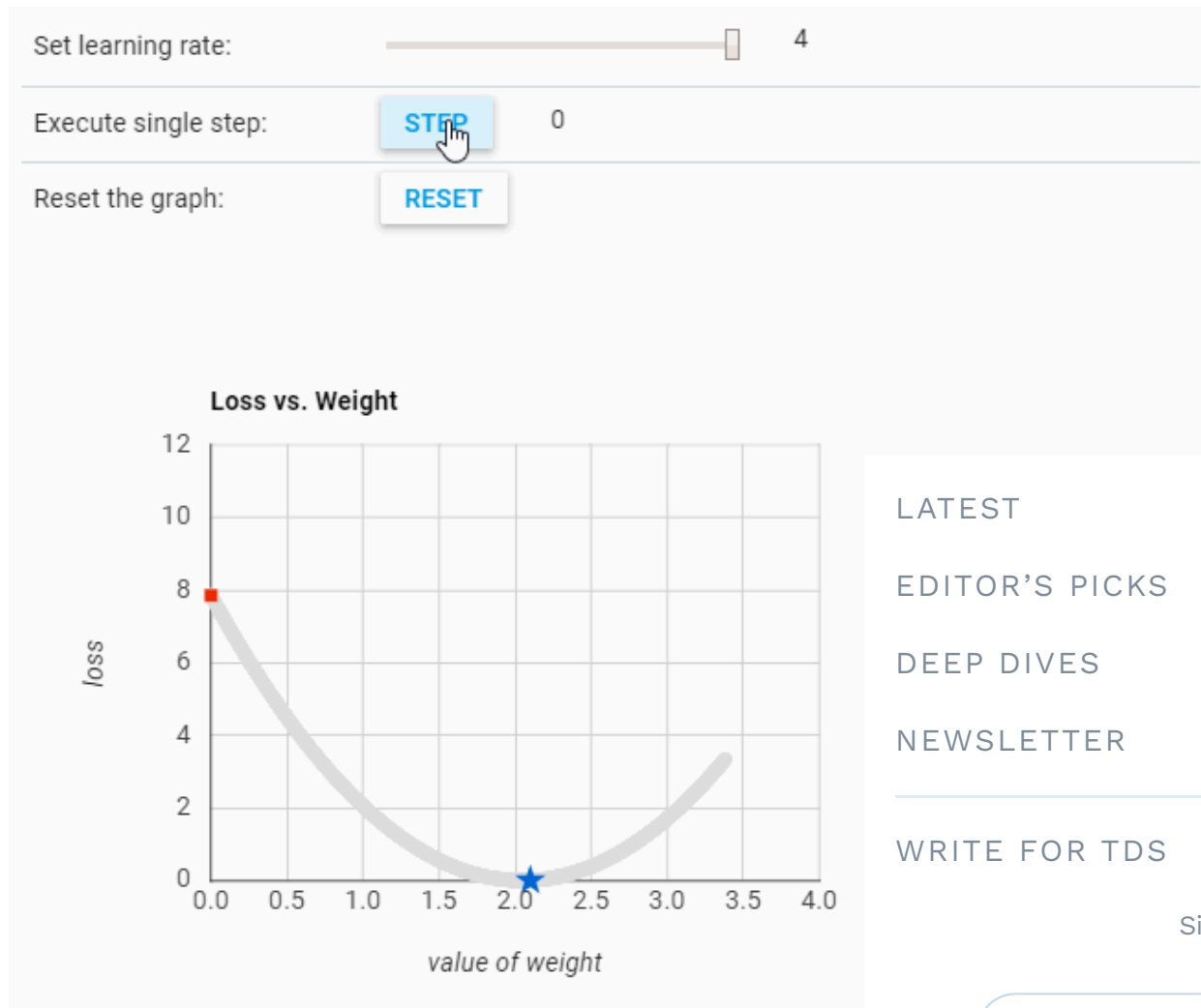
WRITE FOR TDS

Sign in

Submit an Article

[Source](#)

We saw that when the learning rate is too low, it takes too many steps to converge. On the other hand, when the learning rate is too high, Gradient Descent fails to reach the minimum, as seen in the visualization below.



[Source](#)

Experiment with different learning rates by visualizing the loss function below.

Optimizing Learning Rate | Machine Learning

Google Developers

Derivatives

Machine learning uses derivatives in optimization. Optimization algorithms like *gradient descent* decide whether to increase or decrease the value of a parameter to increase or decrease any objective function.

If we are able to compute the derivative of a function, we know in which direction to proceed to minimize it.

Primarily we shall be dealing with two concepts from calculus :

- **Power Rule**

Power rule calculates the derivative of a variable with a power.

If we have a function like

$$f(x) = x^n$$

, then

$$\frac{\partial f(x)}{\partial x} = nx^{n-1}$$

Example : Find the derivative of the function $f(x)$ w.r.t. x where

$$f(x) = 3x^5$$

$$\frac{\partial f(x)}{\partial x} = 15x^4$$

- **Chain Rule**

The **chain rule** is used for calculating the derivative of composite functions. The chain rule can also be written using Leibniz's notation as follows:

If a variable z depends on the variable y , which in turn depends on the variable x , so that y and z are dependent on each other, then z , via the intermediate variable of y , depends on x as well. This is called the chain rule and is mathematically written as,

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Let us understand it through an example:

if

$$y = x^2$$

and

$$x = z^2$$

then, the derivative of **y** w.r.t **z** can be calculated with **chain rule** as follows:

$$\frac{\partial y}{\partial z} = \frac{\partial y}{\partial x} \cdot \frac{\partial x}{\partial z}$$

$$\frac{\partial y}{\partial x} = 2x$$

$$\frac{\partial x}{\partial z} = 2z$$

Hence,

$$\frac{\partial y}{\partial z} = 2x \cdot 2z$$

- Using the **Power** and **Chain** Rule for deriv calculate how the Cost function changes and c. This deals with the concept of **par** which says that if there is a function of t then to find the partial derivative of that one variable, treat the other variable as c be more clear with an example:

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article

Partial Derivatives

Say for instance, we have a function:

$$f(x, y) = x^4 + y^7$$

partial derivative of the function w.r.t 'x' will be :

$$\frac{\partial f}{\partial x} = 4x^3 + 0$$

treating 'y' as a constant

And partial derivative of the function w.r.t 'y' will be :

$$\frac{\partial f}{\partial y} = 0 + 7y^6$$

treating 'x' as a constant

Calculating Gradient Descent

Let us now apply the knowledge of these rules to our original equation and find the derivative of the Cost Function w.r.t to both 'm' and 'b'. Revising the equation :

$$J_{m,b} = \frac{1}{N} \sum_{i=1}^N (Error_i)^2$$

For simplicity, let us get rid of the summation part. The summation part is important, especially with **Stochastic gradient descent (SGD)** vs. **batch gradient descent**. During the **batch gradient descent**, we look at the error of all the training examples at once, whereas in **stochastic gradient descent**, we look at each error at a time. However, to keep things simple, we will assume that we are looking at the error at a time.

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)

$$\frac{\partial J}{\partial m} = 2 \cdot \text{Error} \cdot \frac{\partial}{\partial m} \text{Error}$$

$$\frac{\partial J}{\partial b} = 2 \cdot \text{Error} \cdot \frac{\partial}{\partial b} \text{Error}$$

Now let's calculate the gradient of Error w.r.t to both m and b :

$$\frac{\partial}{\partial m} \text{Error} = \frac{\partial}{\partial m} (Y' - Y)$$

$$\frac{\partial}{\partial m} \text{Error} = \frac{\partial}{\partial m} (mX + b - Y)$$

constants

$$\frac{\partial}{\partial m} \text{Error} = X$$

$$\frac{\partial}{\partial b} \text{Error} = \frac{\partial}{\partial b} (Y' - Y)$$

$$\frac{\partial}{\partial b} \text{Error} = \frac{\partial}{\partial b} (mX + b - Y)$$

$$\frac{\partial}{\partial b} \text{Error} = 1$$

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Plugging the values back in the cost function it with the learning rate:

Submit an Article

$$\frac{\partial J}{\partial m} = 2 \cdot \text{Error} * X * \text{Learning Rate}$$

Determines the direction to minimize the Error

Determines how large a step to take

$$\frac{\partial J}{\partial b} = 2 \cdot \text{Error} * \text{Learning Rate}$$

This **2** in this equation isn't that significant since what we have a learning rate twice as big or half

let's get rid of it too. So, Ultimately, this entire article boils down to two simple equations representing the equations for Gradient Descent.

$$\frac{\partial J}{\partial m} = \text{Error} * X * \text{Learning Rate}$$

$$\frac{\partial J}{\partial b} = \text{Error} * \text{Learning Rate}$$

$$\text{Since } m = m - \delta m$$

$$\text{Since } b = b - \delta b$$

$$m^1 = m^0 - \text{Error} * X * \text{Learning Rate}$$

$$b^1 = b^0 - \text{Error} * \text{Learning Rate}$$

m^1, b^1 = next position parameters; m^0, b^0 = current parameters

Hence, to solve for the gradient, we iterate through points using our new m and b values and calculate partial derivatives. This new gradient tells us the direction of our cost function at our current position and how we should move to update our parameters. The update is controlled by the learning rate.

Conclusion

The point of this article was to demonstrate how gradient descent works. We used gradient descent as an optimization strategy for linear regression. By finding the line of best fit to measure the relationship between heights and weights. However, it is important to note that the linear regression example has been used for simplicity but can be used with other Machine Learning techniques.

[LATEST](#)
[EDITOR'S PICKS](#)
[DEEP DIVES](#)
[NEWSLETTER](#)
[WRITE FOR TDS](#)
[Sign in](#)
[Submit an Article](#)

WRITTEN BY

Parul Pandey

See all from Parul Pandey

Data Science

Derivatives

Gradient Descent

Machine Learning

Towards Data Science

Share This Article



LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

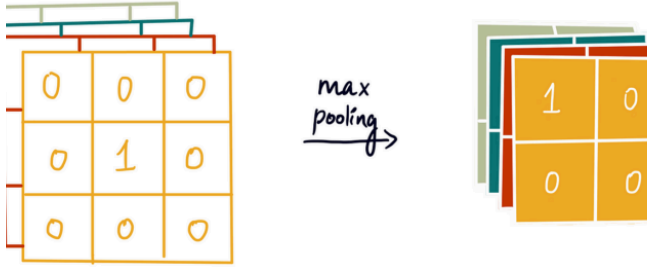
Sign in

Towards Data Science is a community
Submit your insights to reach our global
earn through the TDS Author Paymer

Submit an Article

Write for TDS

Related Articles



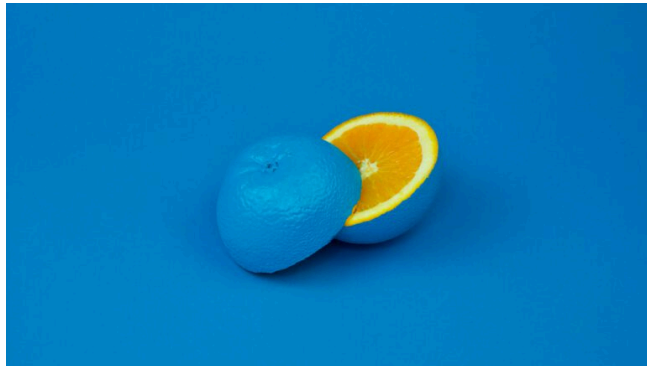
ARTIFICIAL INTELLIGENCE

Implementing Convolutional Neural Networks in TensorFlow

Step-by-step code guide to building a Convolutional Neural Network

Shreya Rao

August 20, 2024 6 min read



DATA SCIENCE

Hands-on Time Series Anomaly Detection using Autoencoders, with Python

Here's how to use Autoencoders to detect signals with anomalies in a few lines of...

Piero Paialunga

August 21, 2024 12 min read

DATA SCIENCE

Solving a Constrained Project Scheduling Problem with Quantum Annealing

Solving the resource constrained project scheduling problem (RCPSP) with D-Wave's hybrid constrained quadratic model (CQM)



ARTIFICIAL INTELLIGENCE

How to Forecast Hierarchical Time Series

LATEST

A beginner's guide to hierarchical time series reconciliation

EDITOR'S PICKS

Dr. Robert Kübler

DEEP DIVES

August 20, 2024

NEWSLETTER

WRITE FOR TDS

[Sign in](#)[Submit an Article](#)

MACHINE LEARNING

3 AI Use Cases for Customer Support (Chatbot)

Feature engineering for unstructured data

Shaw Talebi

August 21, 2024



Luis Fernando PÉREZ ARMAS, Ph.D.

August 20, 2024 29 min read

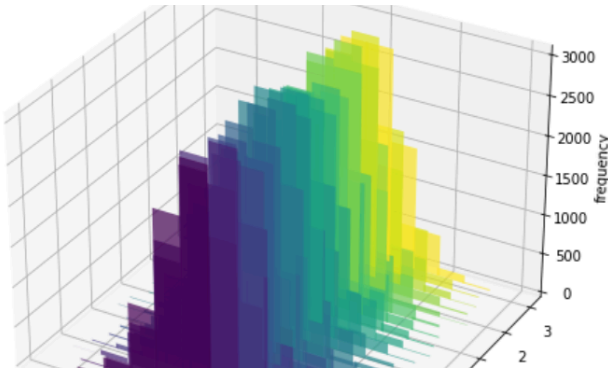
DATA SCIENCE

Back To Basics, Part Uno: Linear Regression and Cost Function

An illustrated guide on essential machine learning concepts

Shreya Rao

February 3, 2023 6 min read



DATA SCIENCE

Must-Know in Statistics: The Bivariate Normal Projection Explained

Derivation and practical examples of this powerful concept

Luigi Battistoni

August 14, 2024 7 min read

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

Sign in

Submit an Article



Your home for data science and AI. The world's leading publ data analytics, data engineering, machine learning, and artifi professionals.

© Insight Media Group, LLC 2026

Subscribe to Our Newsletter

LATEST

EDITOR'S PICKS

DEEP DIVES

NEWSLETTER

WRITE FOR TDS

[Sign in](#)

[Submit an Article](#)